new_alignment

```
"""
NewAlignment 난독화 클래스
-----------------------------------
목적:
    - 난독화된 APK 파일을 정렬(alignment)하여 최적화.
    - APK 내부 데이터가 4바이트 경계에 정렬되도록 처리.
    - APK 실행 효율 향상 및 일부 분석 툴 탐지 회피 가능.

동작 방식:
    1. Obfuscation 객체의 align_obfuscated_apk() 호출.
    2. APK 내부 모든 파일의 정렬(alignment) 수행.
    3. 오류 발생 시 로그 기록 및 예외 발생.

특징:
    - 단순 난독화 후 최적화용 트리비얼 오브퍼케이터.
    - APK 실행 성능이나 일부 정적 분석 회피에 도움.
    - 직접 Smali나 XML을 수정하지 않음.

주의 사항:
    - 난독화 과정 후 APK에만 적용 가능.
    - align_obfuscated_apk()에서 예외 발생 시 APK 처리 실패.
"""
```

예외 처리 분석

obfuscapk -use-appt2 -o NewAlignment -o Rebuild -o NewAlignment -o NewSignature -d /workdir/newalignment.apk /workdir/calender.apk 를 진행했을 때 발생한 문제

```
PS C:\apktest_newalignment> docker run --rm -it -v "C:\apktest_newalignment:/workdir" obfuscapk --use-aapt2 -o NewAlignment -o
 Rebuild -o NewAlignment -o NewSignature -d /workdir/newalignment.apk /workdir/calendar.apk
>>
27/10/2025 10:03:41> [ERROR][obfuscapk.tool.Zipalign][align()] Unable to find file "/workdir/newalignment.apk"
27/10/2025 10:03:41> [ERROR][obfuscapk.obfuscation][align_obfuscated_apk()] Error during apk alignment: Unable to find file "/workdir/newalignment.apk"
27/10/2025 10:03:41> [ERROR][yapsy_loaded_plugin_NewAlignment_1.new_alignment.NewAlignment][obfuscate()] Error during execution of "NewAlignment" obfuscator: Unable to f
ind file "/workdir/newalignment.apk"
/newalignment.apk"
Traceback (most recent call last):
  File "/Obfuscapk/obfuscapk/main.py", line 153, in perform_obfuscation
    (obfuscator_name_to_function[obfuscator_name])(obfuscation)
  File "/Obfuscapk/obfuscapk/obfuscators/new_alignment/new_alignment.py", line 20, in obfuscate
    obfuscation_info.align_obfuscated_apk()
  File "/Obfuscapk/obfuscapk/obfuscation.py", line 620, in align_obfuscated_apk
    zipalign.align(self.obfuscated_apk_path)
  File "/Obfuscapk/obfuscapk/tool.py", line 223, in align
    raise FileNotFoundError('Unable to find file "{0}"'.format(apk_path))
FileNotFoundError: Unable to find file "/workdir/newalignment.apk"
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/Obfuscapk/obfuscapk/cli.py", line 189, in <module>
    main()
  File "/Obfuscapk/obfuscapk/cli.py", line 171, in main
    perform_obfuscation(
  File "/Obfuscapk/obfuscapk/main.py", line 153, in perform_obfuscation
    (obfuscator_name_to_function[obfuscator_name])(obfuscation)
  File "/Obfuscapk/obfuscapk/obfuscators/new_alignment/new_alignment.py", line 20, in obfuscate
    obfuscation_info.align_obfuscated_apk()
  File "/Obfuscapk/obfuscapk/obfuscation.py", line 620, in align_obfuscated_apk
    zipalign.align(self.obfuscated_apk_path)
  File "/Obfuscapk/obfuscapk/tool.py", line 223, in align
    raise FileNotFoundError('Unable to find file "{0}"'.format(apk_path))
FileNotFoundError: Unable to find file "/workdir/newalignment.apk"
```

로그 분석 ⇒ 어떤식으로 예외가 처리되는지 확인함

순서	파일	클래스 / 함수 (역할 비유)	수행하는 동작 (오류 발생 시)	오류를 던지는 대상 (상위 호출자)
1 (시작)	`tool.py`	`Zipalign.align` (실무자)	`subprocess` 실행 실패 $\rightarrow$ **오류 최초 발생 (raise)**	`obfuscation.py`
2	`obfuscation.py`	`Obfuscation.align_obfuscated_apk` (중간 관리자)	`tool.py` 의 오류를 잡고(catch) $\rightarrow$ 로그 남기고 $\rightarrow$ **다시 던짐 (raise)**	`new_alignment.py`
3	`new_alignment.py`	`NewAlignment.obfuscate` (팀장)	`obfuscation.py` 의 오류를 잡고(catch) $\rightarrow$ **"NewAlignment" 플러그인 로그** 남기고 $\rightarrow$ **다시 던짐 (raise)**	`main.py`
4	`main.py`	`perform_obfuscation` (CEO / 메인 엔진)	`new_alignment.py` 의 오류를 잡고 (catch) $\rightarrow$ **"Critical" 로그 (Traceback)** 남기고 $\rightarrow$ **다시 던짐 (raise)**	`cli.py`
5	`cli.py`	`main` (터미널 실행기)	`main.py` 의 오류를 **잡지 않음 (No**	파이썬 인터프리터

			try...except)	
6 (종료)	파이썬 인터프리터	(최종 실행 주체)	처리되지 않은 예외 발견 $\rightarrow$ **터미널에 Traceback 출력** $\rightarrow$ **프로그램 중단**	(사용자 터미널)

NewAlignment 클래스

obfuscapk> src > obfuscapk > obfuscators> new_alignment> new_alignment.py

```
import logging

from obfuscapk import obfuscator_category
from obfuscapk.obfuscation import Obfuscation



    ##  NewAlignment 클래스:
     ##   - 난독화된 APK 정렬(alignment) 수행
       ## - 실행 성능 최적화 및 일부 분석 회피


class NewAlignment(obfuscator_category.ITrivialObfuscator):

    def __init__(self):
        ## 로거 설정
        self.logger = logging.getLogger(
            "{0}.{1}".format(__name__, self.__class__.__name__)
        )
        #상위 클래스 초기화
        super().__init__()

#난독화 실행
    def obfuscate(self, obfuscation_info: Obfuscation):
        self.logger.info('Running "{0}" obfuscator'.format(self.__class__.__name__))

        try:
            #Obfuscation 객체를 통해 APK 정렬 수행
            #Obfuscation 클래스의 obfuscation_info 객체의 align_obfuscated_apk() 메서드에게 실제 정렬 작업 위임
            obfuscation_info.align_obfuscated_apk()
        except Exception as e:
            #오류 로그 기록 및 예외 발생
            self.logger.error(
```

```
            'Error during execution of "{0}" obfuscator: {1}'.format(
                self.__class__.__name__, e
            )
        )
        raise

    finally:
        #사용된 난독화기록에 추가
        obfuscation_info.used_obfuscators.append(self.__class__.__name__)
```

def obfuscate을 보면 Obfuscation 내의 align_obuscated_apk() 매소드에게 실제 정렬 작업을 위임하고 있다.

Obfuscation 내부의 align_obfuscated_apk() 매소드

Obfuscapk> src > obfuscapk > obfuscation.py 내부의 class

```
def align_obfuscated_apk(self) -> None:
    # This method must be called AFTER the obfuscated apk has been signed.

    # The obfuscated apk will be aligned with zipalign.
    zipalign: Zipalign = Zipalign()
    if self.is_bundle:
        return

    try:
        zipalign.align(self.obfuscated_apk_path)
    except Exception as e:
        self.logger.error("Error during apk alignment: {0}".format(e))
        raise
```

zipalign: Zipalign = Zipalign() 을 보면 obfuscation.py의 이 매서드도 실제 작업을 직접 수행하지 않고, Zipalign이라는 또 다른 클래스에 작업을 위임하고 있다.

---

tool.py > Zipalign.align() 매서드

Ziipalign클래스 내부의 align() 매소드는 이미 난독화와 재빌드(Rebuild)가 완료된 APK를 받아서 이를 안드로이드 SDK의 zipalign이라는 외부 명령어를 실행해서 newAlignment 진행

Obfuscapk>src>obfuscapk>tool.py>Zipalin > align()

```
class Zipalign(object):
    def __init__(self):
        self.logger = logging.getLogger(
            "{0}.{1}".format(__name__, self.__class__.__name__)
```

```
        )

        if "ZIPALIGN_PATH" in os.environ:
            self.zipalign_path: str = os.environ["ZIPALIGN_PATH"]
        else:
            self.zipalign_path: str = "zipalign"

       #shutil.which으로 zipalign 경로 미리 확인
        full_zipalign_path = shutil.which(self.zipalign_path)

        # Make sure to use the full path of the executable (needed for cross-platform
        # compatibility).

        #zipalign 경로 확인
        if full_zipalign_path is None:
            #없으면 RuntimeError 발생
            raise RuntimeError(
                'Something is wrong with executable "{0}"'.format(self.zipalign_path)
            )

        else:
            #zipalign 경로 있으면 전체 경로 저장
            self.zipalign_path = full_zipalign_path


#obfuscation.py의 lign_obfuscated_apk()에서  zipalign.align(self.obfuscated_apk_path)을 통해 호출되는 부분
    #apk_path: 정렬할 apk 파일 경로(이미 Rebuild 된 apk)
    #출력: str. zipalign 명령어가 성공정으로 실행되었을 때 내뱉는 "성공 로그" 반환
    def align(self, apk_path: str) -> str:


        #zipaglin을 실행하기 전에, 입력받은 apk 파일 존재 여부 확인
        if not os.path.isfile(apk_path):
            self.logger.error('Unable to find file "{0}"'.format(apk_path))
            #파일이 없으면 raise 발생시켜 프로그램 즉시 안전하게 중단시킴
            raise FileNotFoundError('Unable to find file "{0}"'.format(apk_path))

        # Since zipalign cannot be run inplace, a temp file will be created.
       # 임시로 사용할 apk 복사본 경로 생성
        apk_copy_path = "{0}.copy.apk".format(
            os.path.join(
                os.path.dirname(apk_path),
                os.path.splitext(os.path.basename(apk_path))[0],
            )
        )
```

```
#이제부터 작업 시작
    try:
        #apk_path(원본 apk 파일)을 apk_copy_path(임시 복사본 )로 복사
        apk_copy_path = shutil.copy2(apk_path, apk_copy_path)

#zipalign 명령어 조립 및 실행

#algin_cmd: subprocess로 실행할 zipalign 명령어 리스트
        align_cmd = [
            self.zipalign_path,
            "-p", #apk 내부의 .so(네이티브 라이브러리) 파일들도 정렬
            "-v", #자세한 실행 로그 출력
            "-f", #출력 파일이 이미 존재하면 강제로 덮어쓰기
            "4", #4바이트 경계로 정렬
            apk_copy_path, #입력 파일: 임시 복사본(예: app.copy.apk)
            apk_path,   #출력 파일: 이미 Rebuild가 진행된(난독화) apk 파일 경로(덮어쓰기) 예: app.
apk
        ] #zipalign은 app.copy.apk을 읽어서, 정렬된 버전을  app.apk에 덮어쓴다.

        self.logger.info('Running align command "{0}"'.format(" ".join(align_cmd)))

        #subprocess로 zipalign 명령어 실제 실행
        #zipalign 명령어가 성공적으로 실행되면(종료코드 0), 그 출력(stdout)을 output 변수에 저장

        output = subprocess.check_output(
            align_cmd, stderr=subprocess.STDOUT
        ).strip()
        return output.decode(errors="replace") #성공로그를 문자열로 반환

    #zipalign 명령어 실행 실패시(check_output이 예외 발생시, 종료코드 1이상 시)
    except subprocess.CalledProcessError as e:
        self.logger.error(
            "Error during align command: {0}".format(
                e.output.decode(errors="replace") if e.output else e #zipalign 오류로그
            )
        )
        raise

    #zipalign 실패 외의 예상치 못한 다른 예외 발생시
    except Exception as e:
        self.logger.error("Error during aligning: {0}".format(e))
        raise

    #오류가 나든 안나든 실행
    finally:
```

```
        # Remove the temp file used for zipalign.
        if os.path.isfile(apk_copy_path):
            os.remove(apk_copy_path) #임시로 만들었던 app.copy.apk 파일이 남아있으면 삭제
```

---

main.py

Obfuscapk> src > obfuscapk> main.py

```
#모든 플러그인을 실행(new_alignment.py등 난독화 파일 호출)
    for obfuscator_name in obfuscator_progress:
        try:
            if interactive:
                obfuscator_progress.set_description(
                    "Running obfuscators ({0})".format(obfuscator_name)
                )
            (obfuscator_name_to_function[obfuscator_name])(obfuscation)
        except Exception as e:
            #new_alignment.NewAlignment.obfuscate()와 같이 처리된 예외가 여기로 전달됨
            #exc_info=True 옵션으로 모든 예외 정보 로그에 기록
            logger.critical("Error during obfuscation: {0}".format(e), exc_info=True)
            raise #프로그램 전체 중단
```

이 코드를 보면 모든 플러그인을 실행시키는데 new_alignment.py와 같은 난독화 파일을 호출하고, 예외 발생시 로그를 기록하고, 상위로 예외를 raise 시킨다

⇒ cli.py

---

cli.py

이 부분에서 try execpt 문, 즉 예외처리를 하는 부분이 없으므로 예외가 오면 프로그램 전체가 중단되고, Traceback 전체가 터미널에 출력되고, 스크립트 실행 즉시 종료된다.

```
#main.py에서 perform_obfuscation() 호출해서 오류 받음
#try except문이 없으므로 예외 발생 시 main() 함수로
    perform_obfuscation(
        arguments.apk_file,
        arguments.obfuscator,
        arguments.working_dir,
        arguments.destination,
        arguments.ignore_libs,
        arguments.interactive,
        arguments.virus_total_key,
        arguments.keystore_file,
        arguments.keystore_password,
        arguments.key_alias,
        arguments.key_password,
```

```
        arguments.ignore_packages_file,
        arguments.use_aapt2,
    )

#main 함수에서 예외처리 없으니 프로그램 전체 중단=>Traceback 전체를 터미널에 출력, 스크립트 실행 즉시  종료
if __name__ == "__main__":
    main()
```

예외 처리 진행

obfuscapk/exceptions.py 만들어서 예외처리 고도화

tool.py의 Zipalign 클래스 수정

고도화 1- zipalign의 경로가 확인되지 않을 때

zipalign의 경로가 발견되지 않을 때(zipalign tool 이 없을 때)

RuntimError ⇒ MissingDepedencyError 를 통해 사용자에게 Android SDK Build-Tools를 설치하라는 메세지로 변경

```
#zipalign 경로 확인
        if full_zipalign_path is None:
            #없으면 RuntimeError 발생

           #이전
            #raise RuntimeError(
             #  'Something is wrong with executable "{0}"'.format(self.zipalign_path)
            #)

            #고도화(zipaling이 없다는 건 Android SDK Build-Tools가 설치되지 않았다는 뜻이므로
            #사용자에게 설치 안내 메시지 제공하는 전용 예외 발생)
            raise MissingDependencyError(
                tool_name="zipalign",
                installation_instructions="Please install the Android SDK Build-Tools "
            )

        else:
            #zipalign 경로 있으면 전체 경로 저장
            self.zipalign_path = full_zipalign_path
```

```python
#zipalign 경로 확인
if full_zipalign_path is None:
    #없으면 RuntimeError 발생


    raise RuntimeError(
        'Something is wrong with executable "{0}"'.format(self.zipalign_path)
    )

    #고도화(zipaling이 없다는 건 Android SDK Build-Tools가 설치되지 않았다는 뜻이므로
    #사용자에게 설치 안내 메시지 제공하는 전용 예외 발생)
   # raise MissingDependencyError(
    #    tool_name="zipalign",
    #   installation_instructions="Please install the Android SDK Build-Tools "
    #)

else:
    #zipalign 경로 있으면 전체 경로 저장
    self.zipalign_path = full_zipalign_path
```

⇒

```python
#zipalign 경로 확인
if full_zipalign_path is None:
    #없으면 RuntimeError 발생


    #고도화(zipaling이 없다는 건 Android SDK Build-Tools가 설치되지 않았다는 뜻이므로
    #사용자에게 설치 안내 메시지 제공하는 전용 예외 발생)
    raise MissingDependencyError(
        tool_name="zipalign",
        installation_instructions="Please install the Android SDK Build-Tools "
    )

else:
    #zipalign 경로 있으면 전체 경로 저장
    self.zipalign_path = full_zipalign_path
```

고도화 2 - zipalign 실패 시 예외 처리(return code로 성공/실패 수동 확인)

원래는 subprocess.CalledProcessError 로 예외처리 => 이제는 ToolExecutionError 로 예외처리

zipalign이 실패했을 때(종료코드 1이상) 발생하는 예외 처리

```
if result.returncode != 0:
        error_details = result.stderr.strip()
        self.logger.error(
            "Zipalign 실행 실패.Details: {0}".format(error_details)
        )
```

```
            raise ToolExecutionError(
                tool_name="zipalign",
                details=error_details)
        #성공 시
        #zipalign 명령어가 성공적으로 실행되면(종료코드 0), 그 출력(stdout) 반환
        return result.stdout.strip()
```

고도화 3 - OS 관련 시스템 호출 오류 시 발생 예외 처리(OSError 사용)

FileNotFoundError(파일이 존재하지 않을 때),PermissionError(접근권한이 없을 때) 등

```
    except(OSError) as e:
        self.logger.error("Error during aligning (file error): {0}".format(e))

        raise # 상위 호출자 obfuscation.align_obfuscated_apk()에게 예외 재발생시켜 알림
```

exceptions.py 생성

```
"""
Obfuscapk 의 예외처리 부재 문제를 해결하기 위한 파일

더 상세한 로그와 안정적인 오류 처리를 하기 위해 작성

"""

class ObfuscapkException(Exception):
    """Obfuscapk 관련 예외의 기본 클래스"""

    pass

#1. 도구 관련 오류
class ToolError(ObfuscapkException):
    """
    apktool, zipalign, jarsigner 등
    외부 도구 실행 중 발생한 오류를 나타내는 예외 클래스"""

    def __init__(self,tool_name: str, message: str):
        self.tool_name = tool_name
```

```
        self.message = message
        super().__init__(f"{tool_name} error: {message}")


class MissingDependencyError(ToolError):
    """
    Obfuscapk 실행에 필요한 외부 도구가 누락된 경우 발생하는 예외 클래스
    예: newalignment 난독화를 진행아는데 zipalign 도구가 설치되지 않은 경우
    """

    def __init__(self, tool_name: str):
        self.tool_name = tool_name
        super().__init__(tool_name,f"'{tool_name}' 을 찾을 수 없습니다.""PATH에 설치 및 추가되었
는지 확인해주세요.")

class ToolExecutionError(ToolError):
    """
    외부 도구 실행 중 0이 아닌 종료 코드를 반화하며 실패했을 때 발생
    예: zipalign이 손상된 apk 파일때문에 실행 실패

    """

    def __init__(self,tool_name:str, details:str):
        self.tool_name = tool_name
        #details는 도구가 stderr로 출력한 실제 오류 메세지
        self.details = details
        super().__init__(tool_name,f"'{tool_name}' 실행 중 오류 발생: {details}")
```

결과 확인

1. MissingDependencyError 테스트

방법>

apk 파일이 있는 곳에서 docker 실행

```
docker run --rm -it -v "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\nowindndroi
d-debug_newalignment:/workdir" obfuscapk --use-aapt2 -o NewAlignment -o Rebuild -o
NewSignature -d /workdir/result.apk /workdir/nowinandroid-debug.apk
```

tool.py의 Zipalign 클래스 바로 위에 예외처리 발생

```
raise MissingDependencyError(tool_name="zipalign(강제 테스트 진행)")
```

결과>

exceptions.py에 아래 클래스가 존재하기에

```
class MissingDependencyError(ToolError):
    """
    Obfuscapk 실행에 필요한 외부 도구가 누락된 경우 발생하는 예외 클래스
    예: newalignment 난독화를 진행아는데 zipalign 도구가 설치되지 않은 경우
    """

    def __init__(self, tool_name: str):
        self.tool_name = tool_name
        super().__init__(tool_name,f"'{tool_name}' 을 찾을 수 없습니다.""PATH에 설치 및 추가되었
는지 확인해주세요.")
```

⇒ 아래 와 같이 MissingDependencyError 발생 확인

```
PS C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\nowindndroid-debug_newalignment> docker run --rm -it -v "C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\nowindn
droid-debug_newalignment:/workdir" obfuscapk --use-aapt2 -o NewAlignment -o Rebuild -o NewSignature -d /workdir/result.apk /workdir/nowinandroid-debug.apk
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "/Obfuscapk/obfuscapk/cli.py", line 6, in <module>
    from obfuscapk.main import perform_obfuscation, check_external_tool_dependencies
  File "/Obfuscapk/obfuscapk/main.py", line 8, in <module>
    from obfuscapk.obfuscation import Obfuscation
  File "/Obfuscapk/obfuscapk/obfuscation.py", line 10, in <module>
    from obfuscapk.tool import Apktool, ApkSigner, Zipalign
  File "/Obfuscapk/obfuscapk/tool.py", line 204, in <module>
    class Zipalign(object):
  File "/Obfuscapk/obfuscapk/tool.py", line 206, in Zipalign
    raise MissingDependencyError(tool_name="zipalign(강제 테스트 진행)")
obfuscapk.exceptions.MissingDependencyError: zipalign(강제 테스트 진행) error: 'zipalign(강제 테스트 진행)' 을 찾을 수 없습니다.PATH에 설치 및 추가되었는지 확인해주세요.
PS C:\Users\82102\Obfuscapk\Obfuscapk 고도화 test apk\nowindndroid-debug_newalignment>
```

결론

newalignment 난독화를 진행 시 zipalign이 없는 경우에 대한 고도화 성공

2. PluginExecutionError테스트

방법>

tool.py의 코드 변경 align_cmd 변경

```
 align_cmd = [
            self.zipalign_path,
            "-p", #apk 내부의 .so(네이티브 라이브러리) 파일들도 정렬
            "-v", #자세한 실행 로그 출력
            # "-f", #출력 파일이 이미 존재하면 강제로 덮어쓰기
```

```
        "-invalid-flag", #존재하지 않는 플래그(옵션) - 테스트용
        "4", #4바이트 경계로 정렬
        apk_copy_path, #입력 파일: 임시 복사본(예: app.copy.apk)
        apk_path,   #출력 파일: 원본 apk 파일 경로(덮어쓰기) 예: app.apk
    ] #zipalign은 app.copy.apk을 읽어서, 정렬된 버전을  app.apk에 덮어쓴다.
```